



**I like the concept of the key management systems and integrating that with some form of quantum simulations in Qiskit will be very strong for my research project. help me understand how I can continue the research on the key management systems, whilst also including research into the quantum protocols, including implementations of existing protocols such as bb84 and b92. But it is critical that I prioritise simulations as I do not have access to quantum communications hardware and my interest lies within the algorithm development side of it.**

A very natural way to combine key-management research with protocol work in BB84/B92 is to treat the project as a **two-layer simulation stack**:

1. a *link layer* in Qiskit where BB84/B92 (and maybe one or two other protocols) are implemented and characterized, and
2. a *network/KMS layer* that uses those link-level models to drive key routing, buffering, and provisioning algorithms. [\[1\]](#) [\[2\]](#) [\[3\]](#) [\[4\]](#)

Below is a concrete way to structure this so it stays simulation-only, is algorithm-heavy, and has a clear research contribution.

### **High-level project shape**

The dissertation can be framed as something like:

"Algorithmic design and evaluation of a key management system for QKD networks informed by realistic, Qiskit-based simulations of BB84 and B92 links." [\[3\]](#) [\[4\]](#) [\[1\]](#)

That naturally splits into:

- **Part A – Protocol simulation (Qiskit):** implement BB84 and B92, add noise/eavesdropping models, and extract metrics such as secret key rate vs distance, QBER vs channel noise, and entropy of the raw key. [\[5\]](#) [\[1\]](#) [\[3\]](#)

- **Part B – KMS and network algorithms (classical sim):** design and simulate a KMS that takes those link metrics as inputs and decides how to store, forward, and allocate keys across a small network. [\[2\]](#) [\[6\]](#) [\[4\]](#)
- **Part C – Integration & evaluation:** show that “protocol-aware” KMS algorithms (that understand BB84/B92 behavior) outperform naive, protocol-agnostic schemes on metrics like key availability, delay, and waste. [\[7\]](#) [\[4\]](#) [\[2\]](#)

## Layer 1: QKD protocol simulations in Qiskit

There is prior work showing that QKD protocols (including BB84 and B92) can be realistically simulated in Qiskit and matched against experimental data, so this is a well-accepted approach. [\[1\]](#) [\[3\]](#)

### 1. Implement BB84 and B92 in Qiskit

- Use standard Qiskit `QuantumCircuit` models to implement state preparation, random basis choice, measurement, and sifting for BB84 and B92. [\[5\]](#) [\[1\]](#)
- Existing open-source repos already implement BB84/B92/E91 in Qiskit’s `QasmSimulator`; these can be used as a starting point and then extended. [\[5\]](#)
- Include explicit eavesdropper modules (intercept-resend, photon number splitting approximations) to see how QBER and key rates change under attack. [\[3\]](#) [\[5\]](#)

### 2. Add realistic noise and loss models

- Introduce depolarizing noise, phase noise, and loss channels that roughly model fiber or free-space links, at least at an abstract level. [\[1\]](#) [\[3\]](#)
- Parameterize distance or attenuation and record how secret key rate and QBER evolve as a function of those parameters. [\[3\]](#) [\[1\]](#)
- For each protocol, produce fitted models of the form  $R_{\text{sec}}(d, \eta, p_{\text{noise}})$  and  $\text{QBER}(d, p_{\text{noise}})$ , where  $d$  is distance,  $\eta$  is detector efficiency, etc. [\[1\]](#) [\[3\]](#)

### 3. Explore or extend beyond BB84/B92 (optional)

- Papers have introduced “High-Yield QKD” by comparing BB84 and B92 and modifying them to increase key rate under certain conditions, all tested in Qiskit. [\[3\]](#)
- A similar approach could be used to define a “KMS-friendly” protocol variant optimized for stable key output for networks. [\[8\]](#) [\[3\]](#)
- If time allows, one extra protocol (e.g., E91 or a multistage protocol) can be added to show that the KMS can handle heterogeneous links. [\[9\]](#) [\[8\]](#)

## Outcome of Layer 1:

A library of protocol simulations that, for each link type (BB84 or B92 under specific parameters), outputs expected key generation statistics that can be abstracted as per-link performance models for the higher-level KMS. [\[4\]](#) [\[1\]](#) [\[3\]](#)

## Layer 2: KMS and QKD network simulation (classical)

For the key-management side, the aim is not to re-invent a full network simulator, but to focus on **algorithms and data structures** for key storage, forwarding, and provisioning, informed by how QKD networks are described in current tools and standards. [\[6\]](#) [\[2\]](#) [\[4\]](#)

### 1. Study existing KMS and QKD network simulators

- QKDNetSim (for NS-3) models QKD networks, with explicit key buffers, KMS logic, and an ETSI-style key provisioning interface; its documentation is a goldmine for understanding how KMS entities and key buffers are modeled in practice. [\[10\]](#) [\[2\]](#) [\[7\]](#)
- QKDNetSim and follow-on work (QKDNetSim+) focus on representing keys as objects with IDs, timestamps, lengths, and states (INIT/READY/OBSOLETE), exactly what a realistic KMS needs. [\[11\]](#) [\[2\]](#)
- There is also work specifically on KMS design for QKD networks, reviewing approaches, requirements, and open challenges in relay, storage, and interface design. [\[12\]](#) [\[4\]](#)

Rather than porting NS-3, the dissertation can take the **abstractions and APIs** from these tools and implement a lighter-weight, custom simulator in Python focused on KMS algorithms. [\[2\]](#) [\[7\]](#) [\[4\]](#)

### 2. Define an abstract KMS model

- Represent each link between two nodes (Alice–Bob, Bob–Charlie, etc.) as a time-varying key source whose rate and error characteristics are taken from the Qiskit-derived models for BB84/B92. [\[2\]](#) [\[1\]](#) [\[3\]](#)
- Implement per-node key buffers with states and metadata similar to QKDNetSim (ID, size, timestamp, state). [\[11\]](#) [\[2\]](#)
- Implement a simple ETSI-style interface where “applications” request keys with parameters like number of bits, lifetime, and authentication requirements. [\[6\]](#) [\[4\]](#) [\[2\]](#)

### 3. Add network and routing aspects

- Model a small network (e.g., 4–10 nodes) with different QKD links (some BB84-based, some B92-based, maybe with different distances or noise levels). [\[4\]](#) [\[6\]](#) [\[2\]](#)
- Implement key relay across trusted nodes: end-to-end keys are constructed by XOR’ing or one-time-padding hop-by-hop keys, as in current QKD network designs. [\[13\]](#) [\[4\]](#)
- Add routing logic that chooses which path to use for a given end-to-end key based on available key stocks and link performance. [\[4\]](#) [\[2\]](#)

## Integration: making the KMS protocol-aware

The real novelty can come from **making the KMS explicitly aware of which quantum protocol runs on each link and how it behaves**, rather than treating every link as a black-box key pipe. [\[8\]](#) [\[4\]](#) [\[3\]](#)

### 1. Link models from Qiskit into KMS

- For each simulated protocol / channel configuration, fit simple parametric models (e.g., polynomial or exponential fits) for secret key rate and QBER vs distance/noise. <sup>[1] [3]</sup>
- In the KMS simulator, instead of hard-coding link capacities, use those fitted models to compute how many fresh bits are likely to arrive in each time slot on each link. <sup>[2] [3] [1]</sup>
- Optionally, feed back different eavesdropping scenarios (simulated in Qiskit) as changes to link behavior, then test whether the KMS can adapt (e.g., re-route traffic away from compromised links). <sup>[5] [4] [3]</sup>

## 2. Protocol-aware scheduling algorithms

Examples of research-grade algorithmic questions:

- **Key routing and allocation:** Given heterogeneous links (BB84 vs B92, different distances), how should the KMS schedule key usage so that high-value applications get keys from “better” links (higher rate, lower QBER)? <sup>[4] [2] [3]</sup>
- **Adaptive path selection:** When BB84 links degrade (e.g., increased noise in the Qiskit model), can the KMS prefer alternative paths that use B92 links or multi-hop routes, and what is the impact on end-to-end key throughput? <sup>[2] [3] [1]</sup>
- **Key aging and refresh policies:** How often should keys be refreshed or discarded, given finite buffers and varying generation rates? <sup>[11] [4] [2]</sup>
- **Multi-tenant policies:** If several “applications” request keys from the same network, how should the KMS allocate finite key material fairly or according to priorities (e.g., banking traffic vs monitoring traffic)? <sup>[6] [4] [2]</sup>

## 3. Comparisons and baselines

- Compare naive, protocol-agnostic policies (e.g., shortest-path routing, FIFO key usage) with protocol-aware policies that use the BB84/B92 models. <sup>[3] [4] [1]</sup>
- Metrics could include key blocking probability, average key delivery latency, buffer overflows, and total delivered secure bits to applications over a fixed time horizon. <sup>[11] [4] [2]</sup>
- This gives a strong, quantitative story: “explicitly modeling BB84/B92 at link level measurably improves key-management decisions at network level.” <sup>[4] [1] [3]</sup>

## Tooling choices: where Qiskit fits and where it doesn't

It is realistic to split tools by layer rather than doing everything in Qiskit. <sup>[14] [15] [1]</sup>

- Use **Qiskit** for the quantum-mechanical layer: protocol logic, noise/eavesdropping behavior, small-scale link simulations. <sup>[5] [1] [3]</sup>
- Use **Python-based classical simulation** for the KMS and network layer (possibly taking inspiration from tools like QKNetSim or newer frameworks such as NuQKD). <sup>[15] [7] [2]</sup>
- There are also general quantum network simulators (NetSquid, SeQUeNCe, QuNetSim, QDNS) described in surveys; these can inform design without being fully adopted. <sup>[16] [14] [15]</sup>

This keeps the project squarely on **algorithm development and simulation**, without requiring any physical quantum communications hardware. <sup>[14] [1] [2]</sup>

## Concrete workplan (algorithm-focused, all simulation)

### Phase 1 – Foundations (literature + prototypes)

- Read a key-management survey for QKD networks to identify open problems and typical KMS functions. <sup>[12] [4]</sup>
- Study at least one Qiskit-based QKD simulation tool or thesis (e.g., BB84/B92 educational tools) to adopt best practices for protocol modeling. <sup>[1] [3]</sup>
- Prototype minimal BB84 and B92 circuits in Qiskit and verify that basic metrics (QBER vs noise) behave as expected. <sup>[5] [1]</sup>

### Phase 2 – Protocol library in Qiskit

- Finish robust BB84/B92 simulators with noise, eavesdropping, and parameter sweeps (distance/attenuation, detector efficiency). <sup>[3] [1] [5]</sup>
- Fit simple analytic or tabular models for link performance (secret key rate and QBER). <sup>[1] [3]</sup>

### Phase 3 – KMS core simulator

- Design and implement a Python KMS simulator with: key buffers, key states, per-application requests, and simple routing over a small network graph. <sup>[11] [2] [4]</sup>
- Integrate the link models from Phase 2 so that each edge in the graph has a realistic, time-varying key generation profile. <sup>[2] [3] [1]</sup>

### Phase 4 – Algorithm development and experiments

- Implement multiple KMS algorithms (naive vs protocol-aware vs adaptive). <sup>[4] [2]</sup>
- Run extensive simulation campaigns to compare algorithms under different network topologies, traffic loads, and attack/noise scenarios imported from Qiskit. <sup>[2] [3] [1]</sup>
- Analyze trade-offs and identify when protocol-aware KMS policies give significant benefits. <sup>[3] [4]</sup>

### Phase 5 – Positioning beyond research

- Map the KMS design and interfaces to emerging standards (e.g., ETSI GS QKD 014, ITU-T Y.38xx) to show how the algorithms could drop into real-world deployments. <sup>[17] [6] [2]</sup>
- Discuss applicability to financial networks, metro QKD deployments, or cloud providers offering “QKD-as-a-Service,” referencing the key-service model in QKDNetSim and similar work. <sup>[18] [19] [2]</sup>

In summary, this direction allows a strong, self-contained simulation-only dissertation: **Qiskit is used where quantum mechanics matters (BB84/B92 behavior), and a custom KMS/network simulator is used where algorithm design and systems thinking dominate**, with a clear story about how protocol-level quantum modeling improves high-level key-management decisions. <sup>[4] [1] [2] [3]</sup>

1. <https://kth.diva-portal.org/smash/get/diva2:1780183/FULLTEXT01.pdf>
2. <https://www.qkdnetstim.info/models/build/html/qkd.html>
3. <https://journal.esrgroups.org/jes/article/view/5277>
4. <https://arxiv.org/html/2408.04580v1>
5. <https://github.com/rafapirotto/QKD-protocols>
6. <https://openqkd.eu/qkd-network-simulator/>
7. <https://github.com/QKDNetSim/qkdnetstim-v2>
8. <https://onlinelibrary.wiley.com/doi/10.1002/cpe.70221>
9. <https://inspirehep.net/literature/2825538>
10. <https://www.qkdnetstim.info>
11. <https://www.sciencedirect.com/science/article/pii/S2352711024000566>
12. <https://qci-cat.at/key-management-system/>
13. [https://www.itu.int/rec/dologin\\_pub.asp?lang=e&id=T-REC-Y.3800-201910-!!!PDF-E&type=items](https://www.itu.int/rec/dologin_pub.asp?lang=e&id=T-REC-Y.3800-201910-!!!PDF-E&type=items)
14. <https://arxiv.org/html/2408.11993v1>
15. <https://advanced.onlinelibrary.wiley.com/doi/full/10.1002/apxr.202400016>
16. <https://github.com/sequence-toolbox/SeQUeNCe>
17. <https://www.global.toshiba/ww/company/digitalsolution/articles/tsoul/tech/t0203.html>
18. <https://www.toshiba.eu/quantum/wp-content/uploads/resources/London-Quantum-Secured-Metro-Net-work.pdf>
19. <https://www.idquantique.com/why-the-financial-sector-is-embracing-a-dual-quantum-safe-strategy/>
20. <https://webthesis.biblio.polito.it/22714/1/tesi.pdf>
21. <https://qukaydee.com>
22. <https://github.com/CreepPork/next-door-key-simulator>
23. <https://www.idquantique.com/quantum-safe-security/products/cerberis-xg-qkd-system/>
24. <https://www.ciscolive.com/c/dam/r/ciscolive/emea/docs/2024/pdf/BRKETI-2445.pdf>